Flat stats vs ability score modifiers (STR 15 = +2)? - what you suggest? in generic fantasy flat is enough, but with more advanced system (in the feature) w will nid modifiers

HP tracking per character? Party sheet? -> whats the differnece?

Skill list: fixed generic (Athletics, Arcana) or system-specific? for generic, we can use generic, but for system specyfic we should have to have option to add specyfiscs

Question: can rules like skills, stats explaenation be writen as human readable language, so LLM will read the human definistion to use it in game? I could then use original rule book for the system to parse/import rules. Or is it bad idea?

New mechanics of dice throw: as we discuss 3 differnet mechanics. Silent GM, User dice roll when asked by GM, etc. Dice should be rolled only when GM/narration requier it -> example: im sneaking to enemy. GM do a dice roll, then player need to do a dice roll test.

proggresion: best idea i got is how in wharhammer fantasy is: player gets exp point for fights, quest, and so on. Then he can spend XP on differnet stuf. Learn abbility, change stats, etc. Different things, cost diffetrent ammount of exp points.

summary/history window: we need to add summary windows (pop-up) that will based on the campaing give summary of what happend, what are the goals, quest, etc.

i dont understand party sheet? what for?

For a **generic fantasy template**, use **flat base stats plus optional derived modifiers**, not modifiers alone. Raw stats are easier to read, easier to generate during character creation, and more portable across systems, while derived modifiers can be computed later for systems that need them; many fantasy systems treat the ability score as the base attribute and the modifier as a secondary value derived from it.[1] [2] [3]

Stats model

So the best structure is not “flat or modifiers,” but **flat first, modifiers optional**. Store `strength: 15` as the canonical stat, and also allow a calculated or stored `modifier: +2` layer for systems that need roll math; this keeps the generic template simple while remaining compatible with more advanced systems later.[2] [1]

A practical shape would be:

```
"stats": {
  "strength": { "value": 15, "modifier": 2 },
  "dexterity": { "value": 12, "modifier": 1 },
  "constitution": { "value": 13, "modifier": 1 }
}
```

For the generic fantasy mode, the LLM can mostly reason from the `value`, while rule-heavy systems can consume the `modifier` directly. That avoids painting the project into a corner later.[3] [1]

HP and party sheet

HP should be tracked per character, because hit points describe the condition of one specific hero, NPC, or monster, not the whole group. A character sheet answers questions like "How wounded is this person?" while a party sheet answers campaign-level questions like "What quests are active?", "What is the shared inventory?", "Where is the group now?", or "What enemies are in the current encounter?"[4] [5]

So the difference is:

Sheet	Holds	Why
Character sheet	HP, stats, skills, equipment, conditions, XP, learned abilities	character-specific state [4]
Party sheet	current location, shared gold, active quests, reputation, known factions, encounter state, travel state	group/campaign memory [4] [6]

If you are running a solo campaign, the party sheet still helps because it becomes the **campaign memory layer**. It is not only for 4-player groups; it is the place for anything that belongs to the world/session rather than one individual character.[6] [4]

Skills

For skills, your instinct is right: use a **fixed generic core** plus optional system-specific extensions. A small generic list makes the base fantasy template stable and predictable, while extensions let you plug in Warhammer, D&D, or other systems later without redesigning the whole data model.[7] [3]

A good generic core could be:

- athletics
- stealth
- awareness
- survival
- lore
- persuasion
- intimidation

- investigation
- medicine
- arcana

Then allow:

```
"skills": {
  "generic": {
    "stealth": 3,
    "awareness": 2
  },
  "system_specific": {
    "channeling": 1,
    "ride": 2
  }
}
```

That gives you a reusable fantasy template and still leaves room for rulebook-specific imports later.[7] [3]

Human-readable rules

Yes — **human-readable rules are a good idea**, but they should not be the only representation. LLMs work well with natural-language rule explanations because they can interpret intent, edge cases, and tone, and that makes imported rules from original books much easier to use as context. Long-form campaign systems also benefit from keeping structured memory and narrative guidance side by side.[5] [7]

The safer design is a **hybrid model**:

```
{
  "stats_schema": { ...structured data... },
  "skills_schema": { ...structured data... },
  "rules_text": {
    "stats": "Strength measures physical power...",
    "stealth": "Stealth is used when moving quietly..."
  }
}
```

Structured fields are for reliable validation, UI, filtering, and automation. Human-readable text is for LLM interpretation, imported rulebook definitions, and better narration. If you store only prose, automation becomes fragile; if you store only numbers, the LLM loses nuance.[5] [3]

Dice mechanics

Your dice idea is strong: **rolls should only happen when the fiction requires uncertainty**, not on every action. That matches good RPG practice, because checks are usually reserved for meaningful risk, uncertainty, or consequence rather than routine narration. A resolution system based on abilities and modifiers is commonly used only when an action is difficult or opposed.[1] [2]

For Phase 3, I would define **three roll modes**:

Mode	Who rolls	Use case
Silent GM	system rolls privately	stealth detection, hidden traps, secret knowledge
Prompted player roll	player rolls after GM request	"Roll Stealth", "Roll Perception"
Auto system roll	backend rolls openly	quick solo mode, API testing, optional automation

This fits your example well: "I sneak toward the enemy" does **not** automatically force a roll. The GM first decides whether the action is uncertain and meaningful; if yes, the system either silently resolves enemy awareness or asks the player for a roll, depending on the mode.[1]

The key rule should be: **fiction first, roll second**. The player describes intent, the GM decides whether a check is needed, then the mechanic triggers.[1]

Progression

Your Warhammer-style idea is better for this project than fixed auto-leveling. A **spendable XP economy** is more flexible, more immersive, and easier to generalize across many fantasy systems than a strict "gain level, get package" model. Story-based rewards and structured quest/objective tracking are especially useful in long campaigns because they preserve continuity and make growth feel tied to play outcomes.[8] [6]

So instead of:

- Level 2 at 300 XP
- automatic bonuses

use:

```
"progression": {
  "xp_total": 120,
  "xp_unspent": 40,
  "purchases": [
    { "type": "skill", "name": "stealth", "cost": 20 },
    { "type": "stat", "name": "dexterity", "cost": 30 }
  ]
}
```

Then each system can define its own upgrade catalog and costs. Generic fantasy can start with a simple catalog like "raise stat," "raise skill," "buy talent," "learn spell," while Warhammer-like systems can later attach more specific advancement rules.[6] [3]

Summary window

Yes, a **summary/history window is essential.** Long-running campaigns lose continuity fast, and campaign-level recap plus quest tracking solves exactly that problem by keeping goals, unresolved threads, and recent events visible in one canonical place. Systems built for long-form campaign memory benefit from a central objective layer and session recap that can be reviewed and updated over time.[6] [5]

I would split it into three views:

View	Purpose
Session recap	what happened recently
Quest log	active, completed, failed goals
World state	NPCs, locations, factions, threats

That popup should be generated from the party sheet/campaign sheet, not only from raw turn history. Raw turns are too long; the summary window should show compressed memory. [8] [6]

What party sheet is for

You said "I don't understand party sheet — what for?" The simplest answer is:

Character sheet = one person. Party sheet = everything shared.[4]

Without a party sheet, shared information gets scattered across turns and character records:

- current location
- current objective
- quest state
- discovered locations
- known NPCs
- hostile factions
- loot owned by the whole group
- who is in the current encounter
- round/order if combat is active

That information does not belong to one character, but the GM and LLM still need it constantly. The party sheet gives the model a compact campaign memory layer and gives the UI a place to show "where we are and what matters now." [4] [6]

Suggested Phase 3 decisions

For the generic fantasy template, I would lock in these choices now:

1. **Stats:** raw values are canonical; modifiers are optional derived helpers.[2] [1]
2. **HP:** always per character; no shared party HP.[4]
3. **Party sheet:** yes, as campaign/world/shared-memory state.[6] [4]
4. **Skills:** generic core + system-specific extension bucket.[7]
5. **Rules:** hybrid format — structured fields plus human-readable rule text.[5]
6. **Dice:** only when fiction requires it; support silent GM, prompted player, and auto-roll modes.[1]
7. **Progression:** spendable XP, not only auto-level packages.[8] [6]
8. **Summary window:** yes, generated from campaign memory, quests, and recap state.[6]

Recommended data shape

A good Phase 3 baseline would be:

```
{
  "character": {
    "stats": {
      "strength": { "value": 12, "modifier": 1 },
      "dexterity": { "value": 14, "modifier": 2 }
    },
    "hp": { "current": 10, "max": 10 },
    "skills": {
      "generic": { "stealth": 2, "awareness": 1 },
      "system_specific": {}
    },
    "progression": {
      "xp_total": 100,
      "xp_unspent": 30,
      "purchases": []
    }
  },
  "campaign": {
    "summary": "The hero entered the ruined monastery...",
    "quests": [
      { "title": "Find the origin of the dzidy", "status": "active" }
    ],
    "world_state": {
      "current_location": "Krwawy Kamień monastery",
      "known_npcs": [],
      "known_factions": []
    },
    "dice_mode": "prompted_player"
  }
}
```

That gives you a clean generic fantasy base without blocking future system-specific overlays. [5] [6]

The next thing to spec should be the **exact JSON schema** for `character.sheet_json` and `campaign` shared state, because almost every other mechanic depends on that structure.

⁂

1. https://rpg.brentnewhall.com/2014/01/a-generic-rpg-system-based-on-ability-scores/
2. https://www.aonprd.com/Rules.aspx?ID=84
3. https://www.ttrpgtraveller.com/ttrpg-design-blog/ability-scores-deciding-what-a-character-is
4. https://goblinpunch.blogspot.com/2018/04/party-sheets.html
5. https://www.jenova.ai/en/resources/ai-rpg-generator
6. https://www.myarchivist.ai/blog/quest-log
7. https://www.amsel-suite.com/article/70/how-to-create-player-character-sheets-for-your-ttrpg-campaign
8. https://www.reddit.com/r/rpg/comments/1f4ixeg/i_made_a_quest_log_for_my_players_and_its_really/
9. https://www.reddit.com/r/rpg/comments/1m934ku/the_party_is_a_character/
10. https://www.reddit.com/r/RPGdesign/comments/boiztf/why_both_ability_scores_and_modifiers/